Université Cadi Ayyad
École Nationale des Sciences Appliquées de Safi
Cycle ingénieur, année universitaire 2026-2027

Module : Théorie des Jeux et Systèmes Multi-Agents
Projet intégrateur n° 6

Dilemme du prisonnier répété : tournoi d'Axelrod et agents LLM

Mini-livrable de la semaine 3 : outils LangChain

Séance du 25 novembre 2026

Réalisé par (groupe 6) :
TSETSE Komla Edem Wilfried (IDIA3)
SADOUR Assia (IDIA3)
MARZAQ Jihane
ZERBAOUI Marwane

Encadré par :
Pr Abdelilah JRAIFI

1. Objet du projet

Le projet n° 6 simule un tournoi d'Axelrod entre stratégies du dilemme du prisonnier répété, mesure l'effet du bruit et de la dynamique du réplicateur, puis fera jouer des agents LLM locaux pour les comparer aux stratégies classiques. La semaine 3 expose les fonctions de la semaine 2 comme outils d'un agent LangChain, appelés d'abord sans modèle de langage avec .invoke(), et teste la boucle de l'agent avec un faux modèle.

2. Travail réalisé

2.1 Organisation de l'équipe

Les rôles restent ceux des semaines 1 à 4. Le point d'avancement de cette semaine est présenté par Marwane.

Membre	Rôle (semaines 1 à 4)	Travail de la semaine 3
Marwane	Développeur de l'agent	outils @tool, prompt système, boucle de l'agent
Jihane	Testeuse	tests hors ligne des outils et de la boucle
Assia	Quant / Python	relecture des outils, notebook 02
Wilfried	Chef de projet et théorie	Gantt réalisé, point d'avancement

2.2 Trois outils @tool plafonnés

Le module agent_tj_sma.py expose tournoi_axelrod, replicateur et seuils_actualisation. Chaque outil a des arguments typés, une docstring qui décrit les paramètres et les conventions, et une sortie JSON dans laquelle les types NumPy sont convertis. Les paramètres coûteux sont plafonnés : 1000 manches, 2000 générations, 20 répétitions et 7 stratégies au plus ; un ajustement est signalé dans un champ d'avertissements. Les stratégies sont désignées par un nom pris dans une liste fermée : un texte qui contient du code est refusé comme nom inconnu, et aucun eval ni exec n'est utilisé. Une valeur invalide renvoie un JSON {"erreur": ...} que l'agent peut lire, et un mauvais type est refusé par le schéma de l'outil avant son exécution.

2.3 Faux modèle scripté et boucle de l'agent

Un faux modèle de chat rejoue des réponses préparées (appels d'outils puis réponse finale) et étiquette chacune de ses sorties « scripté ». Il permet de tester hors ligne la boucle construite avec create_agent de LangChain 1.5 : l'outil est appelé, son résultat revient au modèle, la réponse est produite. Ces tests prouvent la mécanique et rien sur la qualité d'un vrai modèle. La suite pytest compte maintenant 41 tests, tous verts : 25 sur les fonctions pures et 16 hors ligne sur les outils et la boucle (valeurs connues, identité avec la fonction pure, reproductibilité à graine fixée, plafonds, entrées invalides, deux appels d'outils enchaînés).

2.4 Notebook 02 et une observation

Le notebook 02 appelle chaque outil avec .invoke() sur des entrées valides puis invalides et rejoue la boucle scriptée avec une trace des appels. Il montre aussi que le réplicateur est sensible au nombre de répétitions du tournoi : à $\varepsilon = 0{,}01$, les parts finales sont rancunier 0,563 et Pavlov 0,437 avec 5 répétitions, contre 0,929 et 0,071 avec 20 répétitions dans le notebook 01. Une conclusion sur la population gagnante exige donc plusieurs graines et un nombre de répétitions suffisant.

3. Choix retenus

Trois choix structurent l'agent. Une erreur de valeur est renvoyée à l'agent sous forme de JSON plutôt que levée, pour qu'il puisse l'expliquer ; un plafond ramène la valeur dans la limite et le dit. Le prompt système impose d'appeler l'outil pour tout chiffre et de rappeler les paramètres utilisés (manches, bruit, graine). Enfin, LangChain 1.5 n'a plus de paramètre verbose pour les agents : la fonction interroger affiche à la place chaque appel d'outil et son résultat.

4. Écarts, difficultés et questions à l'encadrant

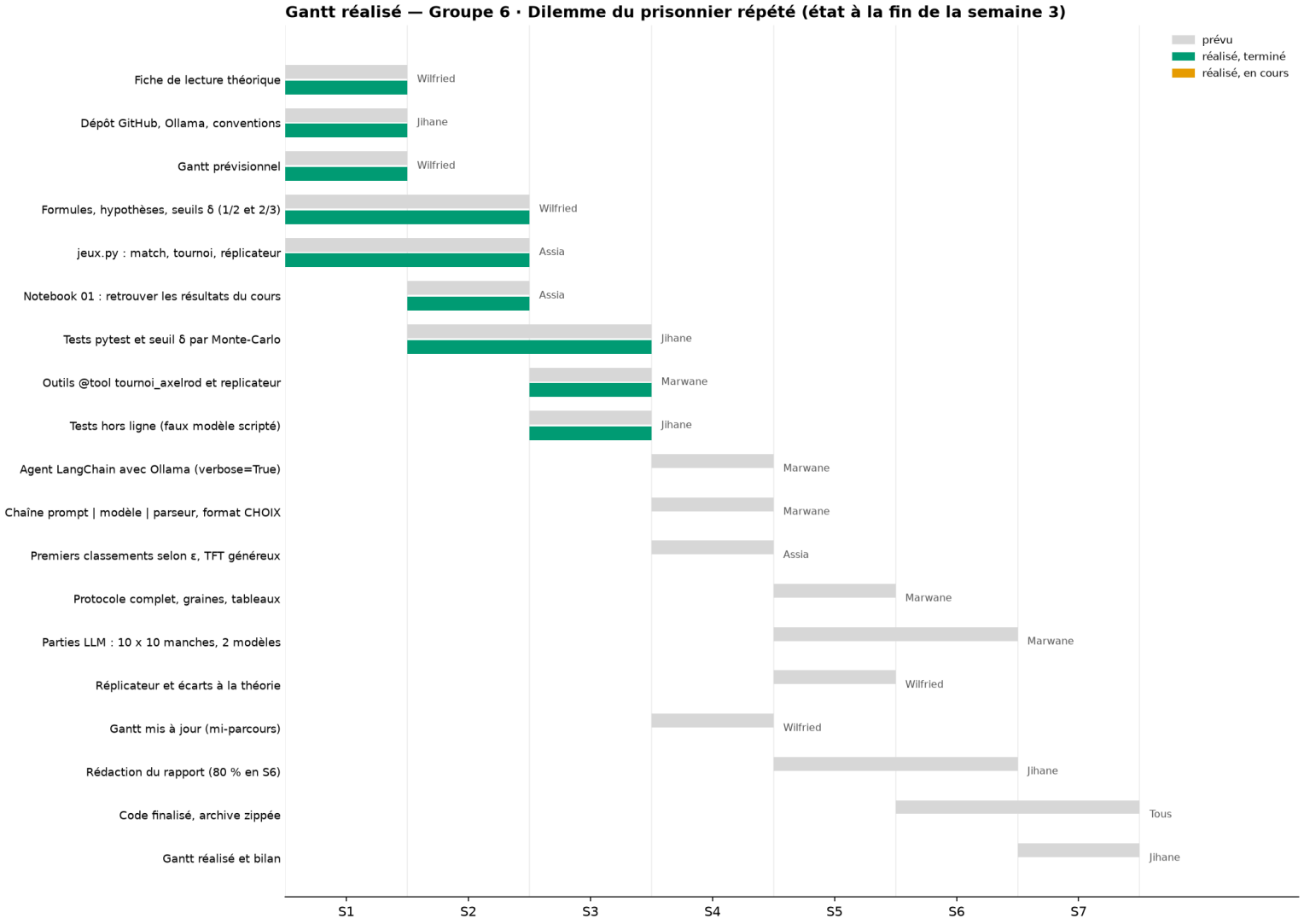
Prévu pour la semaine 3 : outils @tool fonctionnels avec .invoke(), premiers tests pytest. Réalisé : les trois outils, le faux modèle, 16 tests hors ligne et le notebook 02. Pas encore fait : aucun essai avec un vrai modèle, ce qui est prévu en semaine 4 ; l'agent n'a donc été exercé qu'avec le faux modèle (scripté). Restent à relever à la séance : les commits de chaque membre ([[à relever avec git shortlog -sn]]) et l'installation d'Ollama chez Assia, Jihane et Marwane ([[à confirmer en séance]]). Difficulté persistante : le code du cours (agent_tj_sma.py) n'est pas dans nos supports et l'agent est reconstruit d'après le guide ([[réponse du professeur]]).

1. Les outils de l'agent du cours renvoient-ils du JSON sous forme de chaîne, comme dans notre reconstruction, ou des objets Python ?

5. Objectifs de la semaine 4

La semaine 4 est la mi-parcours. L'agent sera testé avec le faux modèle puis avec Ollama (llama3.1 et qwen2.5), en traçant chaque appel d'outil. Le joueur LLM sera construit avec la chaîne prompt, modèle, parseur et le format « CHOIX : COOPERER » ou « CHOIX : TRAHIR », les réponses sans cette ligne étant comptées à part. Les premières expériences (classement selon ε, TFT généreux) seront lancées, les rôles tourneront et le Gantt prévu sera comparé au réalisé.

Annexe : diagramme de Gantt réalisé à la fin de la semaine 3



Prévu en gris, réalisé en vert (terminé) ou orange (en cours), à la fin de la semaine 3.